Architecture 1001: x86-64 Assembly

Intel Registers

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

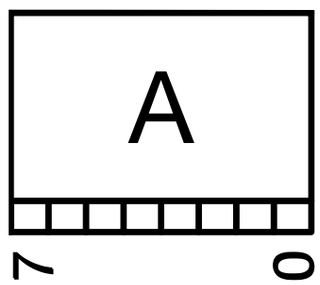
Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

Architecture - Registers

- Registers are small memory storage areas *built into the processor* (still volatile memory)
- Intel has 16 “general purpose” registers + the instruction pointer which points at the next instruction to execute
  - But 2 of the 16 are not *that* general
- On x86-32, registers are 32 bits wide
- On x86-64, registers are 64 bits wide

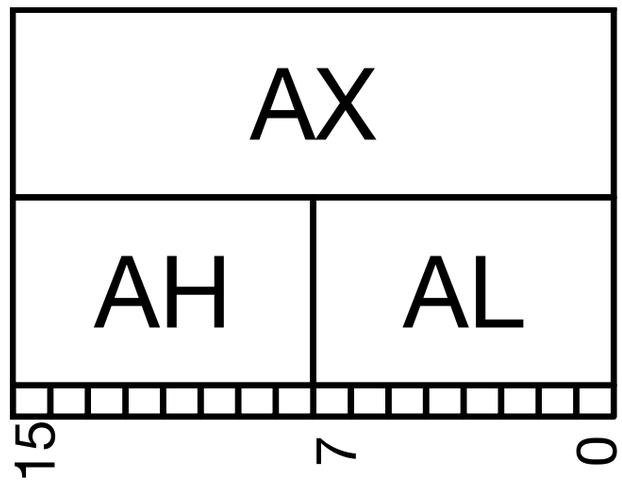
Intel Register Evolution

8-bit *8008*



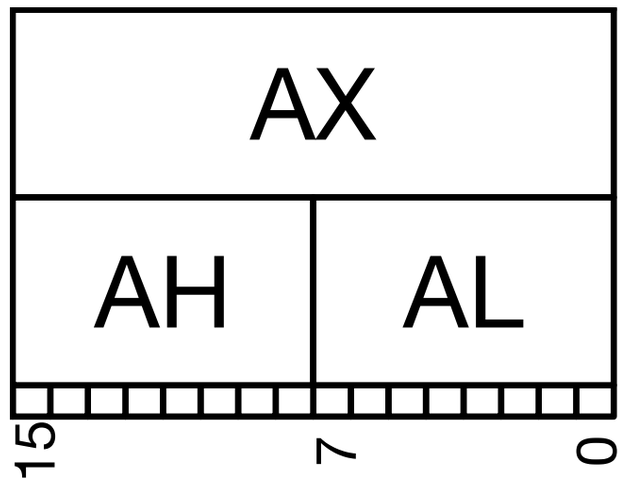
Intel Register Evolution

16-bit 8086



Intel Register Evolution

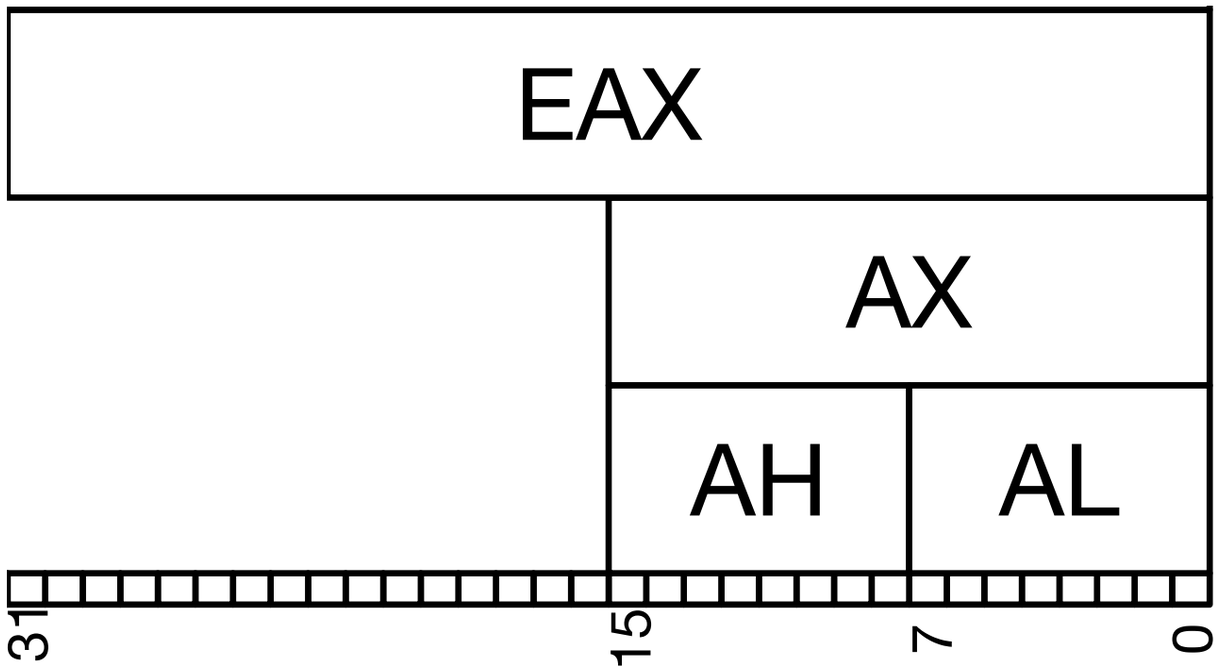
16-bit 8086



A e**X**tended
A **H**igh
A **L**ow

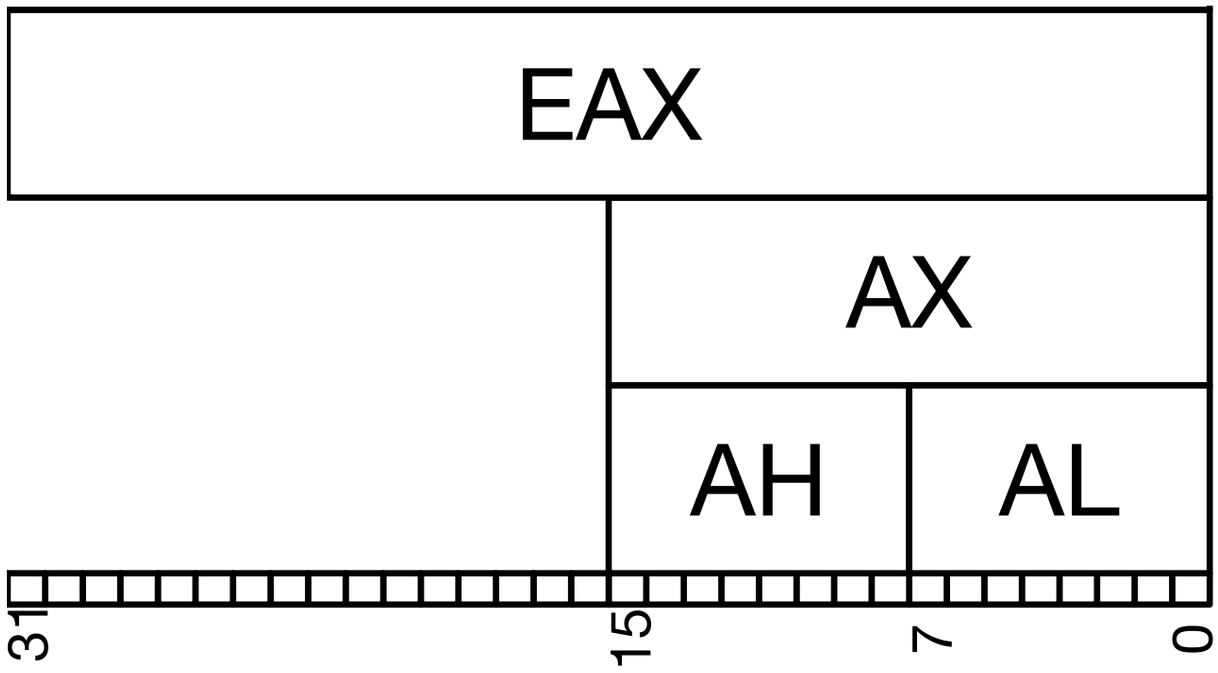
Intel Register Evolution

32-bit 80386



Intel Register Evolution

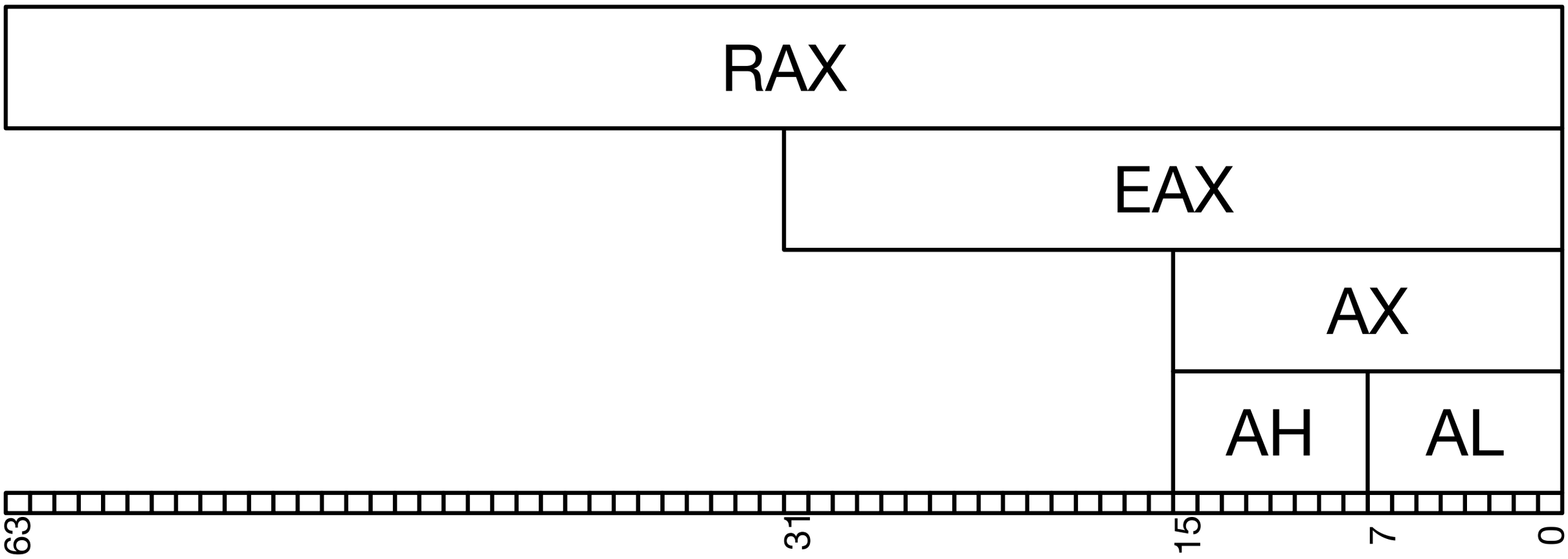
32-bit 80386



Extended AX

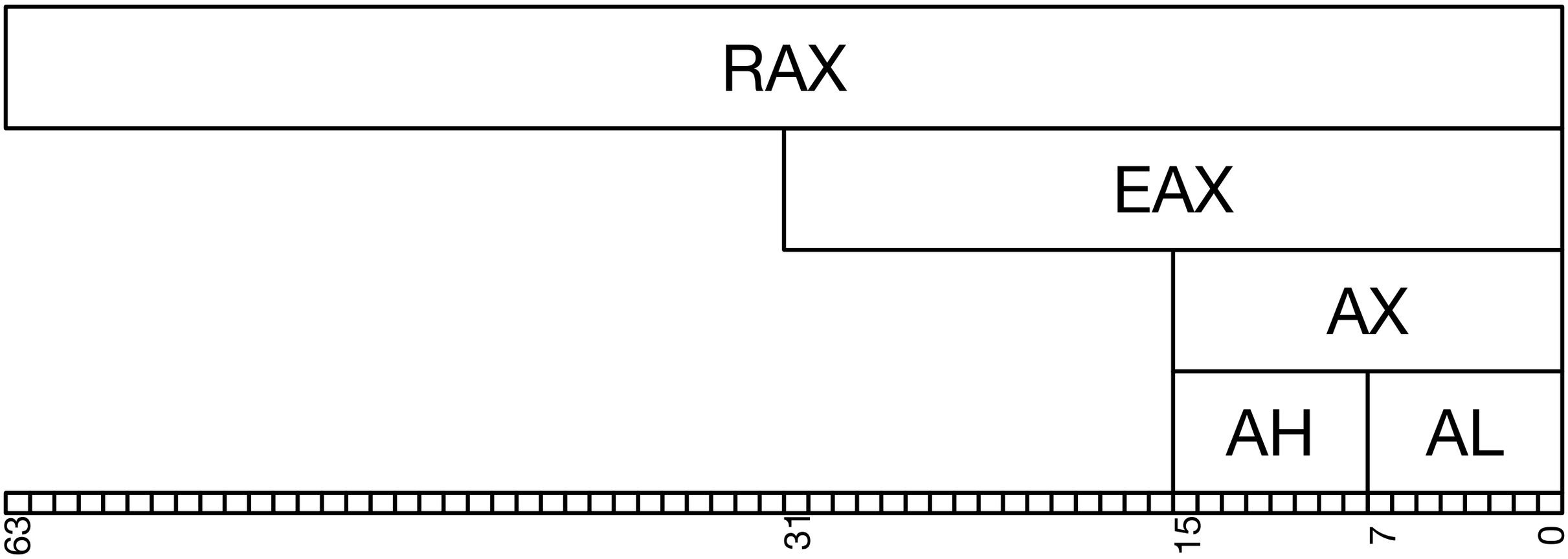
Intel Register Evolution

64-bit AMD Opteron / Intel Pentium 4



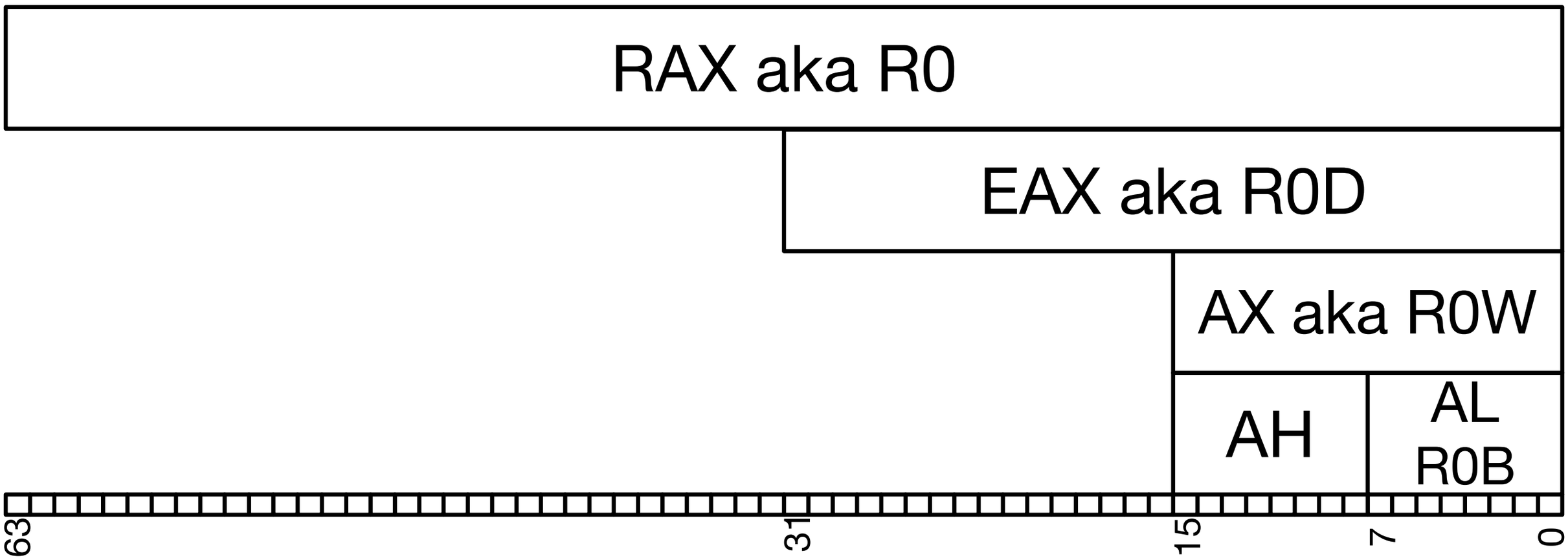
Intel Register Evolution

64-bit AMD Opteron / Intel Pentium 4



Intel Register Evolution

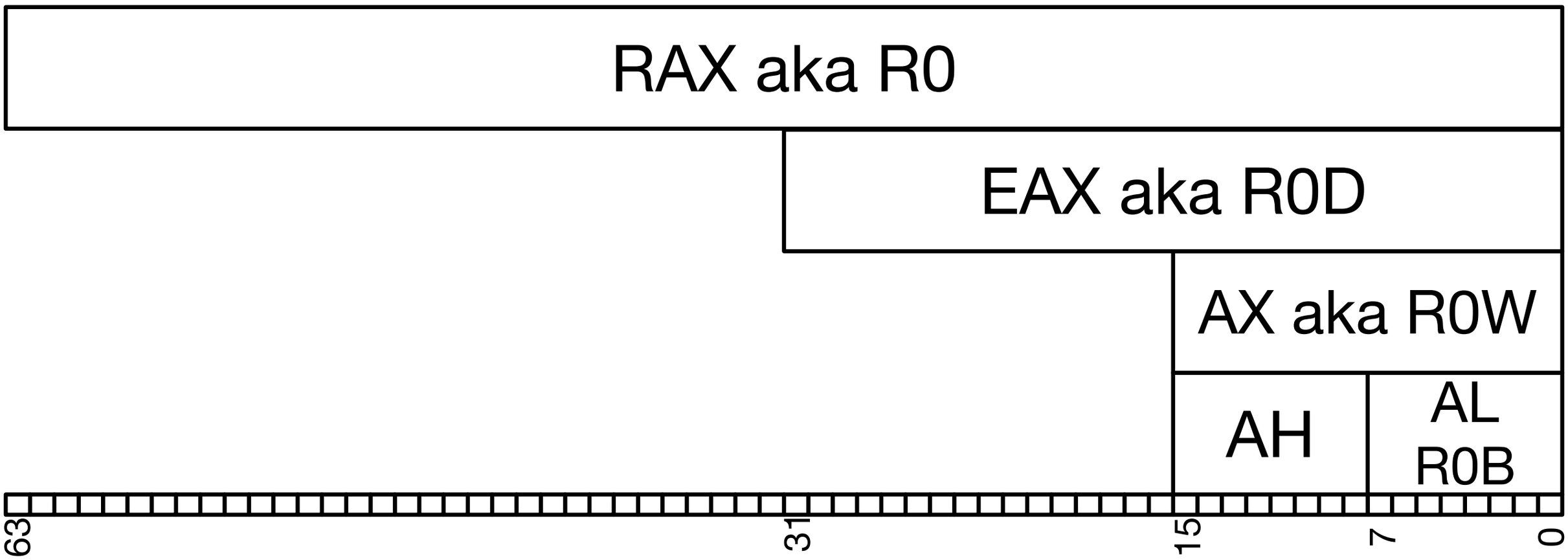
64-bit AMD Opteron / Intel Pentium 4



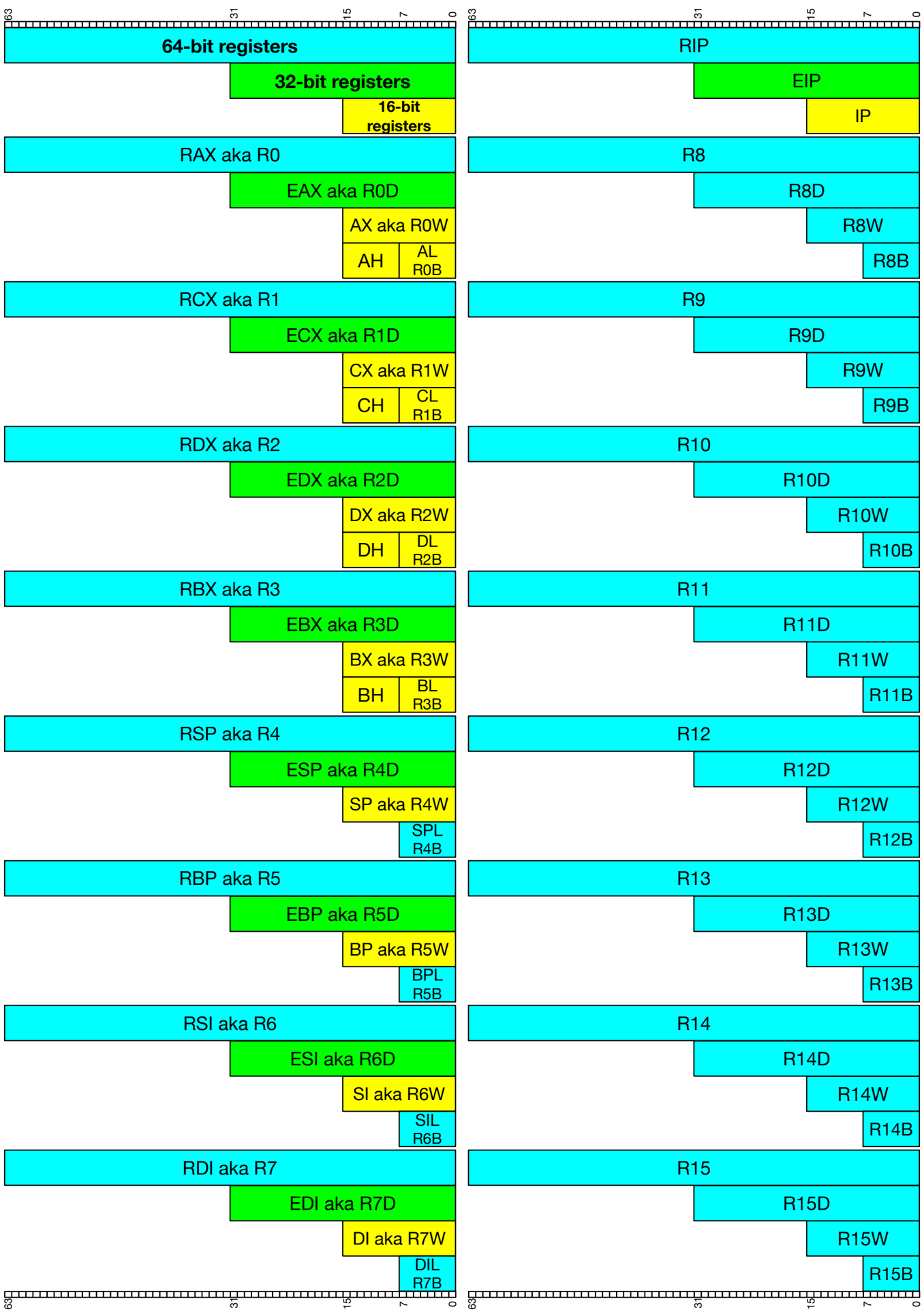
Really-wide AX
Or just
Register 0

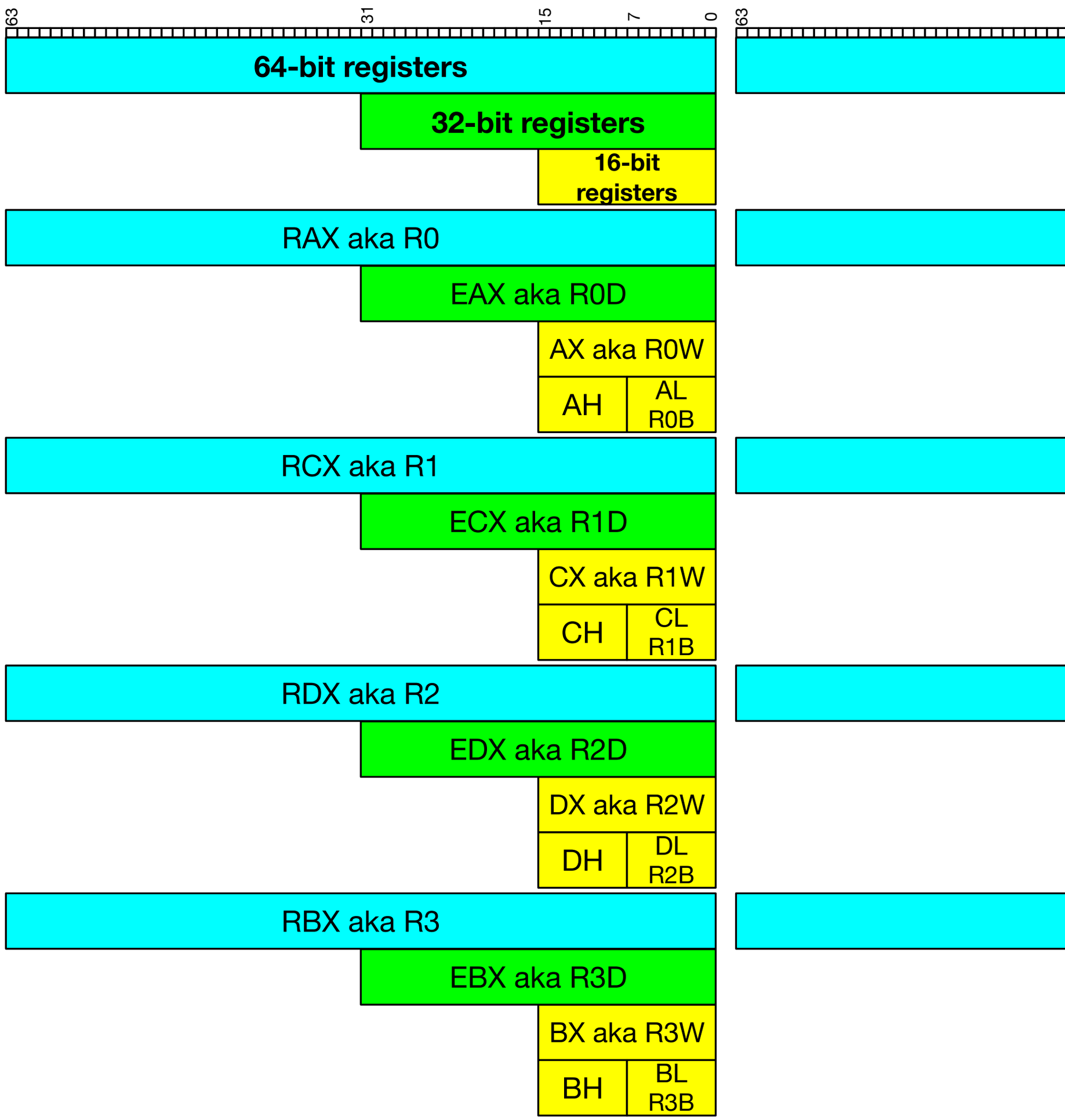
Intel Register Evolution

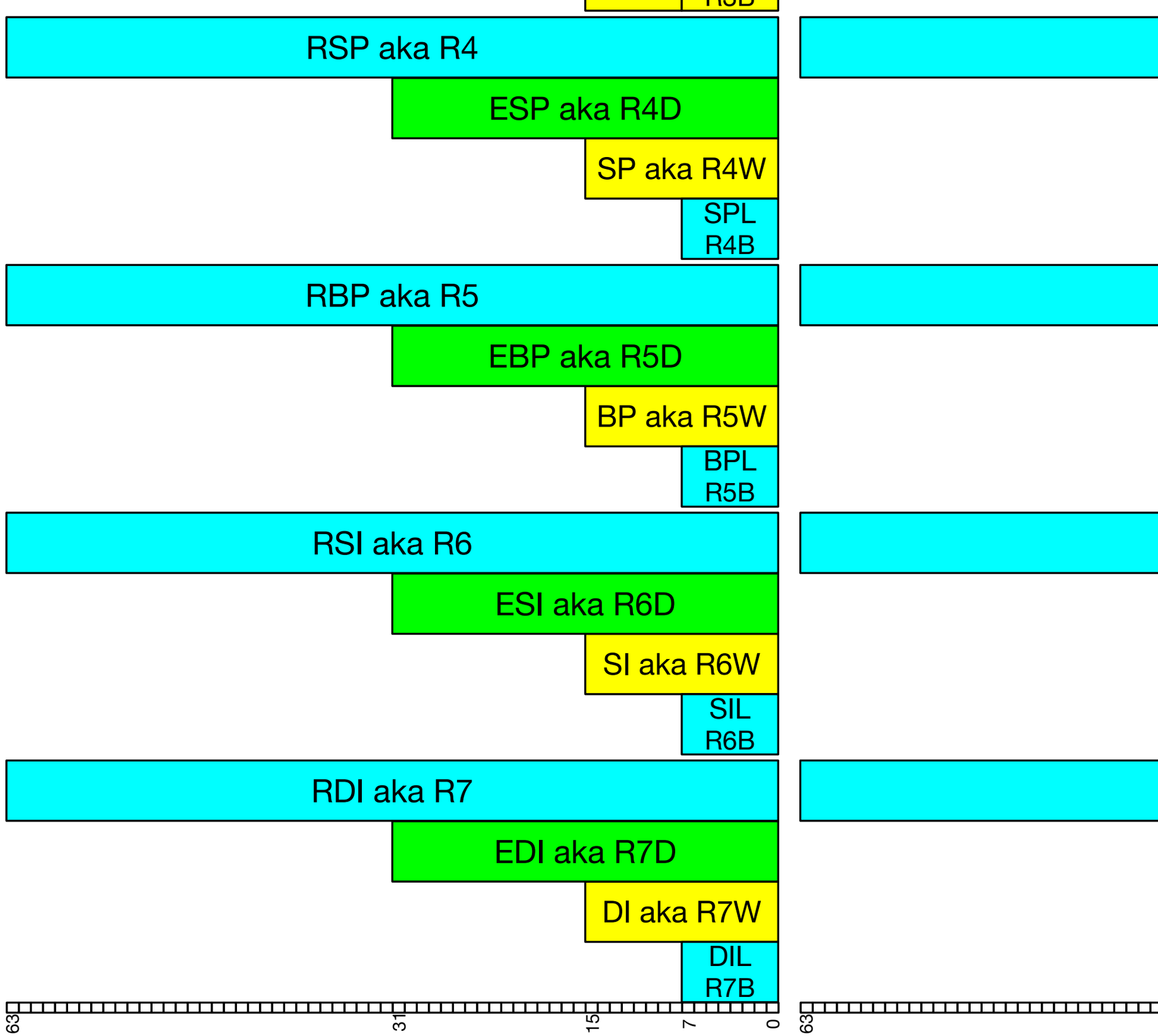
64-bit AMD Opteron / Intel Pentium 4

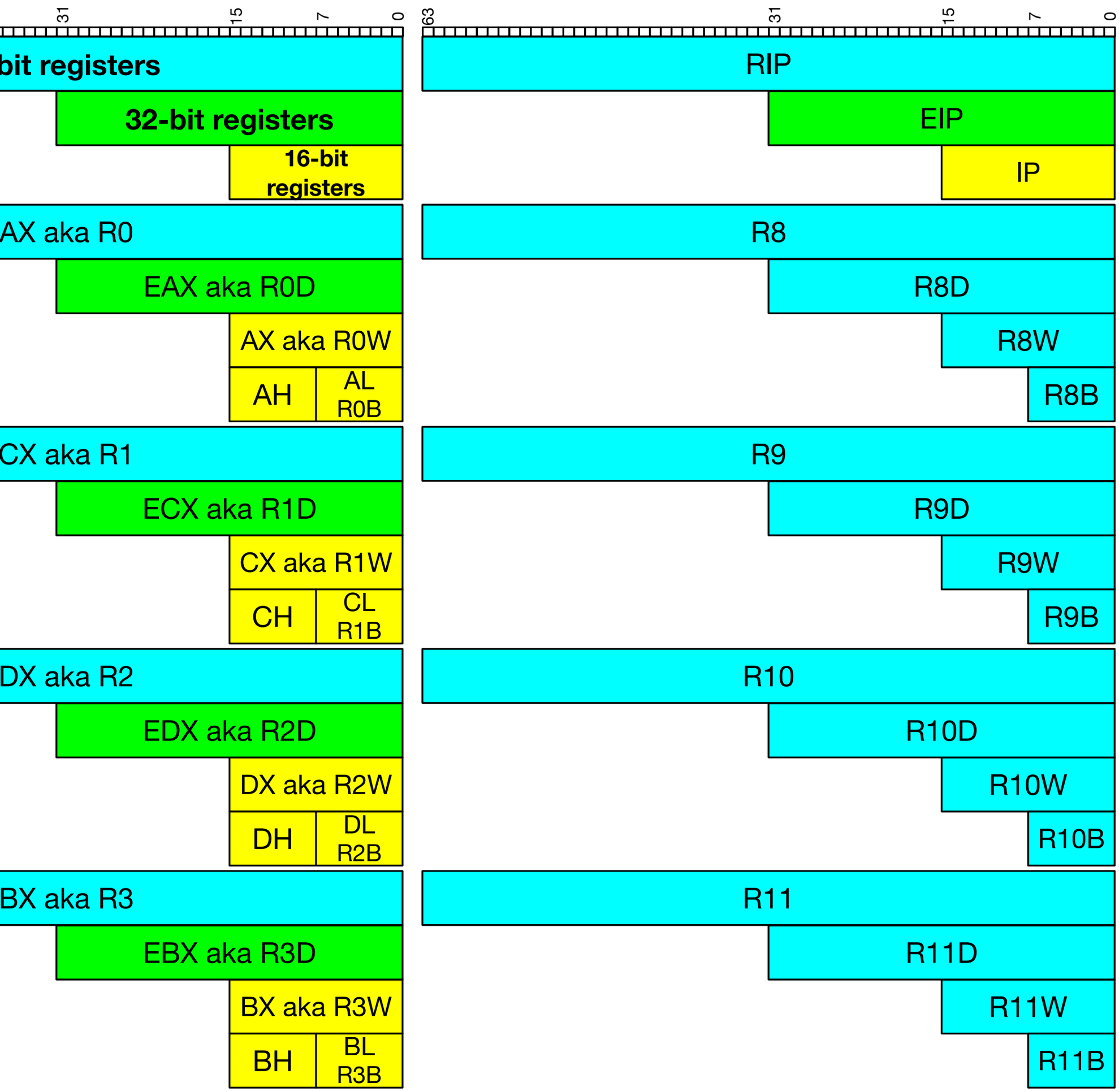


Really-wide AX	R0-**D**(ouble)Word
Or just	R0-**W**ord
Register 0	R0-**B**yte


63
31
15
7
0
64-bit registers
32-bit registers
16-bit registers
RAX aka R0
EAX aka R0D
AX aka R0W
AH
AL R0B
RCX aka R1
ECX aka R1D
CX aka R1W
CH
CL R1B
RDX aka R2
EDX aka R2D
DX aka R2W
DH
DL R2B
RBX aka R3
EBX aka R3D
BX aka R3W
BH
BL R3B
RSP aka R4
ESP aka R4D
SP aka R4W
SPL R4B
RBP aka R5
EBP aka R5D
BP aka R5W
BPL R5B
RSI aka R6
ESI aka R6D
SI aka R6W
SIL R6B
RDI aka R7
EDI aka R7D
DI aka R7W
DIL R7B
RIP
EIP
IP
R8
R8D
R8W
R8B
R9
R9D
R9W
R9B
R10
R10D
R10W
R10B
R11
R11D
R11W
R11B
R12
R12D
R12W
R12B
R13
R13D
R13W
R13B
R14
R14D
R14W
R14B
R15
R15D
R15W
R15B


63
31
15
7
0
63
64-bit registers
32-bit registers
16-bit registers
RAX aka R0
EAX aka R0D
AX aka R0W
AH
AL R0B
RCX aka R1
ECX aka R1D
CX aka R1W
CH
CL R1B
RDX aka R2
EDX aka R2D
DX aka R2W
DH
DL R2B
RBX aka R3
EBX aka R3D
BX aka R3W
BH
BL R3B


RSP aka R4
ESP aka R4D
SP aka R4W
SPL
R4B
RBP aka R5
EBP aka R5D
BP aka R5W
BPL
R5B
RSI aka R6
ESI aka R6D
SI aka R6W
SIL
R6B
RDI aka R7
EDI aka R7D
DI aka R7W
DIL
R7B
63
31
15
7
0
63


31
15
7
0
bit registers
32-bit registers
16-bit registers
AX aka R0
EAX aka R0D
AX aka R0W
AH
AL R0B
CX aka R1
ECX aka R1D
CX aka R1W
CH
CL R1B
DX aka R2
EDX aka R2D
DX aka R2W
DH
DL R2B
BX aka R3
EBX aka R3D
BX aka R3W
BH
BL R3B
63
31
15
7
0
RIP
EIP
IP
R8
R8D
R8W
R8B
R9
R9D
R9W
R9B
R10
R10D
R10W
R10B
R11
R11D
R11W
R11B

Intel Register Conventions 1

These are Intel's suggestions to compiler developers (and assembly handcoders). Registers don't have to be used these ways, but if you see them being used like this, you'll know why. But I simplified some descriptions. I also color coded as **GREEN** for the ones which we will actually see in *this* class (as opposed to future ones), and **RED** for not.

- **RAX** – Stores function return values
- **RBX** – Base pointer to the data section
- **RCX** – **C**ounter for string and loop operations
- **RDX** – I/O pointer

Intel Arch v1 Section 3.4.1 - General-Purpose Registers, page 3-11
Also MS's conventions: https://docs.microsoft.com/en-us/cpp/build/x64-software-conventions?view=msvc-160

Intel Register Conventions 2

- **RSI** – **S**ource **i**ndex pointer for string operations
- **RDI** – **D**estination **i**ndex pointer for string operations
- **RSP** – **S**tack (top) **p**ointer
- **RBP** – Stack frame **b**ase **p**ointer
- **RIP** - Pointer to next instruction to execute (“**i**nstruction **p**ointer”)